

29-08-21 Ex 4:- Build a simple feed forward neural network to recognize hand written characters.
Aim:- Build a simple neural network that can recognize handwritten characters.

Objective:-

1. Get the hand written image and labels.
2. Prepare the images so the computer can understand them.
3. Make a small neural network to learn patterns in the images.
4. Teach the network using many examples.
5. Check how well the network can recognize new images.

Pseudocode:-

START

Load training and test image

Prepare the images (flatten and normalize)

Create a neural network with:

- Input layer (pixels)
- one or two hidden layers
- output layer (number of characters)

Repeat for several times:

Give a batch of images to the network

Calculate how wrong the network is (loss)

Update the network so be better (train)

Test the network on new images

Show how accurate it is.

End

Output :-

Epoch 1	A
Accuracy :	90.29%
Epoch 2	
Accuracy :	95.64%
Epoch 3	
Accuracy :	97.06%
Epoch 4	
Accuracy :	97.70%
Epoch 5	
Accuracy :	98.50%

Result :-

The feed forward neural network, trained on MNIST achieved around 95%. Approximately on the test data. This indicates the model learned to recognize most handwritten digits correctly. Although simple, it provides a solid baseline for digit classification task.

Accuracy :- $\frac{\text{True positive} + \text{True negative}}{\text{Total Instance}}$

F1 Score $\frac{\text{True positive} + \text{True negative} + \text{False positive} + \text{False negative}}{\text{Total Instance}}$

F1 Score = $\frac{\text{True positive} + \text{True negative}}{\text{Total Instance}}$

Precision

14/08/25 Exp:- 4:-

Build a simple feed forward neural network to recognize handwritten characters:-

Important terms:-

transforms.to_tensor():- Convert images from PIL format (0-255 pixel values) to pytorch tensors (0-1, floats).

datasets..MNIST():- Loads the MNIST datasets.

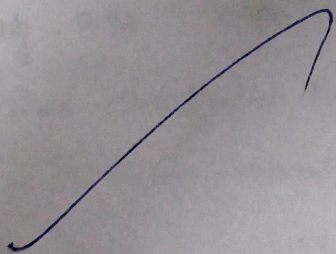
train = True/False:- Specifies training or test dataset.

DataLoader:- Divides data into batches of size 32.

Shuffles training data for randomness; test data

isn't shuffled.

nn.Module:- Base class for all neural networks.



1. Sigmoid Function

$$f(x) = \frac{1}{1+e^{-x}}$$

- output range: (0,1)
- Pros: Useful for probability outputs.
- Cons: Causes vanishing gradients.

2. Hyperbolic Tangent (Tanh)

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- output range: (-1,1)
- Pros: Stronger gradients than Sigmoid.
- Cons: Still suffers from vanishing gradient.

3. Rectified Linear Unit (ReLU)

$$f(x) = \max(0, x)$$

- output range: [0, ∞)
- Pros: fast, reduces vanishing gradient, widely used.
- Cons: Dying ReLU problem.

4. Leaky ReLU:

$$f(x) = \begin{cases} x & x \geq 0 \\ 0.01x & x < 0 \end{cases}$$

fixed dying ReLU by allowing small negative slope.

5. Softmax

$$f(x_j) = \frac{e^{x_j}}{\sum_j e^{x_j}}$$

- Convert raw outputs into probability distributions.
- Commonly used in multi-class classification.